Reasoning RL, Rollouts, and Verifiable Rewards

pig7selene

September 5, 2026

Abstract

Reasoning-oriented post-training changes the feedback interface available to a language model. In domains such as mathematics and programming, a sampled solution can often be checked by an executable or rule-based procedure rather than ranked only by a human preference. This chapter treats a Rollout as an autoregressive reasoning trajectory, distinguishes Outcome Reward, Process Reward, learned Reward Models, and deterministic verifiers, and explains why multiple samples can support both online policy optimization and inference-time selection. It develops the idealized success probability of Best-of-N sampling, clarifies the role of Pass@k and self-consistency, and identifies the credit-assignment, diversity, verification, and curriculum conditions that limit reasoning RL.

1 Introduction

Chapters 13–16 developed post-training around preferences, learned Reward Models, PPO, DPO, and GRPO. Those methods accept several kinds of feedback, but generic preference data leaves a difficult question unresolved: how can a system obtain a reliable scalar signal for a newly generated, long solution? Reasoning tasks sometimes offer a narrower but powerful answer. A numerical answer may be checked against a known value, a program can be executed against tests, and a formal object can be checked against declared rules. The resulting signal is often called a **verifiable reward**.

The important shift is not that every reasoning task has a perfect automatic judge. It is that correctness can be operationalized for some tasks more reproducibly than a broad human-style preference. This makes it practical to generate new candidate solutions, verify or score them, and use the outcomes for online policy optimization. DeepSeekMath is one example of group-based RL on mathematical tasks with reproducible outcome checks [1]. This chapter develops the reusable ideas behind that setting without treating any one reasoning model as a template for all systems.

The central dataflow is

$$\text{prompt} \rightarrow \text{Rollouts} \rightarrow \text{evaluate} \rightarrow \text{select or update}. \tag{1}$$

The two endings in Equation 1 must remain distinct. At **inference time**, a fixed policy spends extra compute generating and selecting candidates for one prompt. At **training time**, scored Rollouts are used to update parameters, after which a new policy generates a new data distribution. PPO and GRPO provide the update machinery; the focus here is the structure and quality of the Rollouts and rewards they consume.

2 Reasoning as Sequential Generation

Let x be a task prompt drawn from a declared distribution $\mathcal{D}_{\text{prompt}}$. A Rollout is one complete sampled continuation of an autoregressive policy,

$$y = (a_0, \ldots, a_{T-1}), \quad y \sim \pi_{\text{old}(\cdot \mid x)}, \quad \pi_{\text{old}(y \mid x)} = \prod_{t=0}^{T-1} \pi_{\text{old}(a_t \mid x, a_{<}t)}. \tag{2}$$

The tokens in y can include intermediate textual steps, tool calls, code, delimiters, and a final answer. A parsing procedure $g(x, y)$ extracts the object to be checked: for example, an integer, a program artifact, or a proof term. The visible sequence can make intermediate work available to a reward procedure, but it is not by itself a guarantee that every textual step faithfully describes an internal computation. Operationally, it is a trajectory of token actions that can be sampled, scored, and trained on.

For one prompt, a rollout procedure commonly draws N candidates,

$$\mathcal{Y}(x) = \{y_i\}_{i=1}^{N}, \quad y_i \sim \pi_{\text{old}(\cdot \mid x)}. \tag{3}$$

The candidates in Equation 3 need not be independent in a strict probabilistic sense. They share a model, prompt, decoding policy, and often a common high-probability mode. Temperature, nucleus sampling, prompt perturbations, and search policy affect their diversity. Nevertheless, collecting several candidates exposes outcomes that one greedy continuation would never reveal. This is useful both when a trainer needs contrast for an update and when a deployed system needs a stronger answer for one fixed prompt.

2.1 Feedback Surfaces and Credit Assignment

An **Outcome Reward** assigns a scalar after the completed response. Write it as $R_{\text{out}(x,y)}$. It may be a learned Reward Model score, a human judgment, or a task score based only on the final artifact. A deterministic verifier is a special operational interface: given a declared checker v, it can produce, for example,

$$R_{\text{ver}(x,y)} = v(x, g(x, y)) \in \{0, 1\}. \tag{4}$$

The binary form in Equation 4 is illustrative, not required. A verifier can return a score, partial test count, compile result, or structured diagnostic. Its key property is reproducibility under a specified parser, checker version, execution environment, and timeout policy. The fact that an evaluator is automatic does not make it complete: a weak parser, incomplete test suite, or incorrect reference answer defines a weak operational target. Training verifiers to rank multiple sampled math solutions is an early example of using generated candidates and an evaluator to improve final selection [2].

A **Process Reward** instead provides feedback at intermediate positions or declared reasoning steps. If the trajectory has step boundaries, a process model can emit $R_{\text{proc}}, t(x, a_{\leq}t)$ before the final answer. The distinction is shown in Table 1.

Feedback surface	What is evaluated	Central limitation
Outcome Reward	A completed response or final artifact.	Sparse: it does not locate the earlier error that caused a failure.
Verifiable Reward	A parsed output under a deterministic or executable checker.	It rewards only the checker specification and can be exploited through its gaps.
Process Reward	Intermediate reasoning prefixes or steps.	It needs trustworthy step boundaries and substantially denser supervision or verification.
Preference-based Reward	A comparison between candidate responses.	It measures the supplied preference rubric, not necessarily task correctness.
Learned Reward Model	A model-predicted scalar for a response or prefix.	It can be miscalibrated or overoptimized outside its training distribution.

Table 1: Feedback mechanisms differ in where they attach supervision, not only in whether their output is a scalar. A system must retain the origin and operational definition of every reward.

Long-horizon reasoning makes credit assignment difficult. Suppose a trajectory receives $R_{\text{ver}} = 0$ after twenty steps. The outcome does not reveal whether the first incorrect transformation, a later arithmetic slip, a malformed final answer, or the verifier itself caused failure. PPO's critic and GAE in Chapter 14 seek token-level estimates from such returns; GRPO in Chapter 16 uses relative outcome performance within a group. Neither automatically turns a final binary label into causal attribution for every preceding token.

Process supervision is motivated by this gap. A Process Reward Model (PRM) can identify an implausible or incorrect intermediate step before the final result, creating denser feedback for training or search. The distinction between process supervision and outcome supervision, along with its data cost and annotation reliability, is central to work on step-level mathematical verification [3]. A process signal is not intrinsically safer than an outcome signal: a PRM can reward locally plausible prose that is globally inconsistent, and optimizing it can create its own shortcut behaviors.

3 Multiple Rollouts, Best-of-N, and Selection

For a fixed prompt, let $C(x, y)$ be the event that a Rollout is correct under an intended task criterion, and let

$$p_x = \mathrm{P}_{y \sim \pi(\cdot \mid x)}[C(x, y) = 1]. \tag{5}$$

If N Rollouts were independent draws with the same success probability p_x, the probability that at least one succeeds would be

$$\mathrm{P}(\text{at least one success}) = 1 - (1 - p_x)^N. \tag{6}$$

Equation 6 is an idealized **coverage** calculation. It says nothing about whether the system can recognize the successful candidate. An exact verifier can select a passing response whenever one occurs. A learned scorer may rank an incorrect response above the correct one, and self-consistency may select a common answer that is not correct. In practice, Rollouts are also correlated, so their effective diversity is lower than the independent-draw assumption suggests. Repeated near-identical samples can make increasing N much less useful than Equation 6 predicts.

Best-of-N means generating N candidates and selecting one using a declared score or verifier. The selection rule might be

$$\hat{y} = \arg\max_{y_i \in \mathcal{Y}(x)} S(x, y_i), \tag{7}$$

where S can be a verifier-derived score, an Outcome Reward Model, a Process Reward aggregation, or another declared ranking function. The selector in Equation 7 is as important as the generator. If S is merely a weak proxy for correctness, allocating more candidates can increase the opportunity to exploit the proxy rather than improve true task success.

3.1 Pass@k and Self-Consistency

Pass@k is an evaluation convention for the probability that at least one of k attempted solutions succeeds. If a finite set of n samples contains c correct solutions, a common estimator is

$$\widehat{\mathrm{Pass@k}} = 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}. \tag{8}$$

The estimator in Equation 8 treats the k attempts as a subset of the observed n samples. It is an evaluation statistic, not a deployed selection policy and not the same object as the idealized model probability in Equation 6. Its interpretation depends on the sampling protocol, correctness checker, and budget. Pass@k became a standard way to report the benefit of repeated program samples in code generation [4].

Self-consistency uses sampled reasoning paths differently. It groups candidates by an extracted final answer and selects the most supported answer, often by a majority or weighted vote. It can help when multiple diverse trajectories reach one correct result and no stronger verifier is available. It is not a proof procedure: shared model biases can make the same wrong answer frequent, and answer parsing can merge or split semantically equivalent outputs. Self-consistency is therefore a selection heuristic whose benefit depends on diversity and answer aggregation, not a substitute for external correctness evidence [5].

4 Training-Time and Inference-Time Compute

Additional Rollouts serve two different budgets. During training, prompts are sampled, multiple responses are generated, and rewards become examples for an optimizer. In a PPO-style loop, the sequence is

$$\text{prompts} \rightarrow \text{Rollouts} \rightarrow \text{rewards} \rightarrow \text{advantages} \rightarrow \text{policy update}. \tag{9}$$

Chapter 14 derives the policy-gradient and PPO components of Equation 9. Chapter 16 replaces a learned Value Model with group-relative advantages when several Rollouts of one prompt provide useful contrast. In both cases, more training Rollouts can improve the statistical signal only if their

rewards are informative and the update remains stable. They also consume generation, evaluation, storage, and optimization compute; they are not free labels.

At inference time, the model parameters stay fixed. The system instead spends a prompt-local budget on multiple candidates, verifier calls, voting, or search,

$$\text{fixed policy} \rightarrow \text{candidate or search budget} \rightarrow \text{selection} \rightarrow \text{one answer}. \tag{10}$$

Training-time Rollout scaling can change future p_x by changing the policy. Inference-time scaling only tries to exploit the current policy's existing success distribution and a selection mechanism for the current prompt. Conflating the two obscures a basic systems choice: a test-time system may improve an answer without learning anything, while an RL update may learn from unsuccessful-looking trajectories yet yield no immediate answer for the sampled prompt.

Search extends Best-of-N when the generator branches conditionally on partial trajectories rather than drawing only independent full completions. A PRM can score prefixes, an outcome verifier can score leaves, and a proposal policy can allocate more candidates to uncertain branches. Such methods make selection more adaptive, but their value depends on the reliability of the intermediate score and on task difficulty. Empirical work on test-time compute finds that no single allocation dominates across difficulty regimes; additional search can also overoptimize an imperfect PRM [6].

5 Failure Modes, Exploration, and Curriculum

Reasoning RL fails when the reward interface and the candidate distribution fail together. Sparse outcome rewards are particularly limiting when almost every Rollout fails: there is little evidence about which direction improves the policy. In GRPO, a group with all identical rewards has no centered within-group signal. The converse problem also occurs: if almost every candidate passes, the training task offers little discrimination. A curriculum can control this by shifting task difficulty as the policy changes, but it is a data-distribution choice that must be versioned and evaluated rather than a universal remedy.

Exploration is equally important. Low temperature or narrow decoding can produce correlated samples whose apparent count hides low effective diversity. Extremely broad sampling can instead produce malformed, off-distribution, or trivially failing candidates. The goal is not maximum textual variation; it is enough variation in valid solution strategies for the reward procedure to distinguish useful behavior. Group size, decoding configuration, maximum length, and stop rules jointly define this trade-off.

Verifiers create their own reward-hacking boundary. A solution may exploit answer formatting, parser ambiguity, an incomplete test suite, a timeout behavior, an uninitialized random seed, or a leaked reference pattern. A model can also learn to emit unnecessarily long reasoning traces if length is indirectly rewarded, or to imitate memorized answer forms without solving new instances. Held-out tasks, adversarial checker tests, length-controlled evaluation, and contamination checks are necessary because a rising verifier pass rate alone proves only improvement under that verifier.

Finally, one must distinguish an outcome's correctness from an explanation's quality. A correct final answer can contain invalid intermediate claims; an incorrect final answer can follow mostly sound steps before one slip. Process supervision can address this distinction only when its step labels or process checks are themselves dependable. It should be treated as a richer feedback interface with its own calibration and distribution-shift risks, not as a guaranteed solution to long-horizon credit assignment.

6 Implementation Contracts

The rollout contract must version the prompt source and split, tokenizer, Chat Template, policy and reference revisions, decoding parameters, maximum response length, stop rules, tool permissions, and parser g. It must retain prompt identifiers, each candidate's group membership, token IDs, action masks, old-policy log probabilities when training, stop reasons, generated length, and any tool traces.

A change to response boundaries or answer extraction changes both the reward and the effective trajectory.

The verification contract must identify the checker implementation, versioned test cases or reference answers, execution image, time, memory, and randomness limits, serialization format, parser failures, score aggregation, and treatment of timeouts or unsupported outputs. It must distinguish a deterministic outcome check from a learned RM and retain their raw component scores separately. Tests should include adversarially formatted candidates, equivalent answers, malformed outputs, and deliberately wrong solutions, so a passing result has a documented meaning.

The optimization contract must state whether the algorithm uses PPO, GRPO, or another update rule; retain its reward-to-advantage transformation; and record the behavior-policy, reference-policy, reward, and verifier revisions. It must also record group size, reward normalization, valid-token denominator, KL estimator, clip range where applicable, optimizer state, precision, gradient controls, and checkpoint state. Evaluation should report task-defined success alongside Rollout count, Pass@k protocol, selector or verifier revision, response length, diversity statistics, reward distribution, and protected held-out results. Chapter 11's resumability requirements apply to every stateful rollout buffer and curriculum cursor.

7 Summary

Reasoning RL operates on the same autoregressive policy structure as other post-training methods, but it often has access to feedback grounded in a task's executable correctness condition. A Rollout is a sampled token trajectory whose final artifact, and sometimes intermediate steps, can be evaluated by an Outcome Reward, a Process Reward, a learned Reward Model, a preference comparison, or a deterministic verifier. These interfaces differ in where they attach signal and in the failure modes they expose.

Multiple Rollouts can improve coverage and selection. Under an unrealistic but informative independence assumption, their chance of containing a correct solution grows as in Equation 6. Real systems also need diversity and a selector that recognizes success, so Best-of-N, Pass@k, self-consistency, and verifier-guided search are related but noninterchangeable tools. More Rollouts during training supply online optimization data; more Rollouts at inference spend a fixed policy's compute budget on one prompt.

The reward interface remains the limiting object. Sparse or uniform rewards weaken learning, weak verifiers invite shortcuts, correlated samples reduce the benefit of scaling, and process signals require their own trustworthy supervision. PPO and GRPO specify how a policy can be updated from Rollouts, but reliable reasoning systems also require careful candidate generation, reward design, selection, curriculum control, and independent evaluation.

References

[1] Z. Shao *et al.*, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024, doi: 10.48550/arXiv.2402.03300.

[2] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems," *arXiv preprint arXiv:2110.14168*, 2021, doi: 10.48550/arXiv.2110.14168.

[3] H. Lightman *et al.*, "Let's Verify Step by Step," in *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2305.20050.

[4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, and others, "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021, doi: 10.48550/arXiv.2107.03374.

[5] X. Wang *et al.*, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2203.11171.

[6] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters," *arXiv preprint arXiv:2408.03314*, 2024, doi: 10.48550/arXiv.2408.03314.